QEC Latency Decoder : Measuring Classical Feedback Latency in Software QEC Decoders, from Simulation to IBM Device Replicas

Swayam Shakya
Faculty of Science & Technology
Algoma University
Brampton, ON, Canada
swayamshakya000@gmail.com

***Abstract*—Quantum error correction only protects information if the classical decoder returns each correction before the qubits lose coherence, yet simulations usually treat this classical step as instantaneous. This study measures it for the smallest useful case, the three-qubit bit-flip code, whose decoder is a four-entry look-up table. The decoder is timed in Python and in C, with the cost of reading the clock measured separately, and the violation rate VR(τ) = P(L > τ) is reported with exact binomial intervals against a 40 µs deadline derived from a 120 µs dephasing time and three syndrome rounds. In the reference run, median decode latencies are 113 ns in Python and 31 ns in C, and VR falls below a 1 percent budget at 268 ns and 46 ns. Across six complete runs totalling 660,000 cycles per implementation, Python missed the deadline 14 times and C once, bounding the violation rates at 0.0036 and 0.00084 percent (95 percent upper limits). The misses come from rare stalls of the host, most likely operating-system preemption, not from decoding: the slowest cycle in each run ranged from 42 µs to 1.39 ms, and no fitted distribution predicts these events. On calibrated replicas of IBM Heron r2 and Eagle r3 processors, the code with real-time feed-forward raises logical fidelity by 1.68 and 1.62 percentage points over a bare qubit, while an explicit decoder wait doubles the logical error after only 7.6 and 12.8 µs. A decoder that meets the 40 µs deadline can therefore still lose most of the code's benefit. All seeded results reproduce bit for bit on the same CPU type from the public repository, where an executed notebook reruns every experiment and re-checks every result. No physical quantum processor was used.**

Keywords*—Quantum error correction, real-time decoding, decoder latency, tail latency, dynamic circuits, reproducibility.

I. Introduction

A quantum error-correcting code protects information only while a loop keeps running: measure the syndrome, decode it, apply the correction (Fig. 1) [1]. Each turn of that loop has to finish before the qubits lose coherence. For superconducting qubits, that happens on a scale of about a hundred microseconds [2]. Fault-tolerance theory assumes the loop keeps up [3], [4]. In a simulation that is easy to grant: the decoder is a function call and costs nothing. In an experiment the decoder is a program running on a real computer, and it can be late.

Real-time decoding is now recognized as a central engineering problem [5]. A recent surface-code experiment below threshold reported an average decoder latency of 63 µs at distance five, against a 1.1 µs cycle time [6]. Most of that work concerns specialized decoders and control stacks. Software decoders on ordinary computers are still widely used for prototyping, teaching, and small experiments, but their timing behavior, and especially their rare slow cycles, is seldom measured. Systems research has long shown that rare slow events, not averages, dominate the behavior of latency-critical software [7].

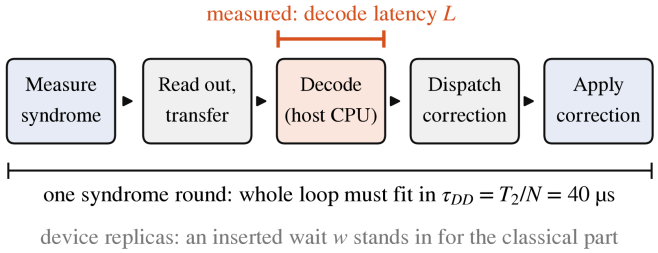


Fig. 1. One round of the error-correction loop. The latency L measured in this study covers the decode step on the host computer. The whole loop must fit in the per-round budget τ_{DD}; on the device replicas, an explicit wait stands in for the classical part.

This study measures that classical step directly, using the simplest code that exercises the full loop. The three-qubit bit-flip code has a four-entry look-up-table decoder whose real work takes a few nanoseconds, so almost everything that appears in its latency distribution comes from the language runtime and the operating system rather than from the decoding algorithm. That makes it a clean probe of the execution environment that every software decoder shares.

The study asks four questions. First, what is the violation rate VR(τ) = P(L > τ) of a software decoder over deadlines from 10 ns to 100 µs, and where does it cross a 1 percent budget (RQ1)? Second, is the latency tail light or heavy (RQ2)? Third, does the code keep its benefit under realistic device noise, and what does feedback latency cost in logical error (RQ3)? Fourth, how much does a compiled C decoder improve on pure Python (RQ4)?

The contributions are an open, reproducible benchmark rather than a new decoder. They comprise timed Python and C decoders with timer-floor controls; exact rare-event statistics for deadline misses; a reproducibility study over six complete

runs on two host CPUs; a dynamic-circuit implementation of the code on calibrated replicas of two IBM processors; and a latency sweep that converts decoder wait into logical error. All code, data, and figures are public, and every number in this paper is regenerated from them.

II. Background and Related Work

A. Quantum Error Correction and the Bit-Flip Code

The three-qubit bit-flip code is the repetition-code building block of Shor's nine-qubit code [8] and a standard teaching example [9]. It corrects any single X error on its data qubits but cannot detect Z errors, so it is not a complete quantum code. Practical schemes such as the surface code measure both kinds of parity on a two-dimensional lattice and tolerate physical error rates up to a threshold of about 1 percent [3], [4]. Repeated rounds of syndrome measurement with real-time correction have been demonstrated on trapped ions [10] and on superconducting processors [11], [12], [6].

B. Real-Time Decoding

A decoder must keep pace with syndrome generation, or a backlog builds up and computation slows exponentially [5]. Decoding large codes is expensive, so much of the literature focuses on algorithms and architectures: union-find decoding in almost linear time [13], faster minimum-weight matching [14], parallel window decoding that trades latency for throughput [15], and dedicated FPGA and ASIC implementations [16]. These works make decoding itself fast. The present study looks at a different part of the loop: the environment a software decoder runs in, which none of those algorithms can remove.

C. Dynamic Circuits

Mid-circuit measurement with classical feed-forward, known as dynamic circuits, has been demonstrated on IBM superconducting processors [17] and is exposed through Qiskit [18]. IBM processors use the heavy-hexagon lattice, designed for low-degree codes with flag qubits [19]. The device experiments here use simulator replicas built from published snapshots of two such processors [20].

D. Tail Latency in Computer Systems

In large online services, responses that are slow only one time in a thousand still shape user-visible latency, and the remedy is to design for the tail rather than the mean [7]. Real-time operating systems address the same problem from the other side, by bounding worst-case scheduling delay; the PREEMPT_RT patch set for Linux is the most widely used example [21]. A QEC decoder running on a general-purpose host inherits none of those guarantees. Measuring its tail is therefore the central task, not a detail.

E. Scope

This study does not propose a decoder and makes no claim about surface-code decoders, whose cost per round grows with code distance. It isolates and measures the host-level latency that every software decoder pays, and it estimates what that latency costs in logical error. No physical quantum processor was used.

III. Problem Formulation

A. The Code and Its Decoder

The logical states are $|0\rangle_L = |000\rangle$ and $|1\rangle_L = |111\rangle$. Two ancilla qubits measure the parities Z_0Z_1 and Z_1Z_2 without measuring the data qubits, so a superposition of logical states survives the check (Fig. 2). Qiskit prints classical bits from right to left, so the two-bit syndrome string reads $(Z_0Z_1)(Z_1Z_2)$. The decoder maps "00" to no correction, "10" to an X gate on qubit 0, "11" to qubit 1, and "01" to qubit 2. If each data qubit flips independently with probability p, the logical qubit fails when two or three flip:

$$P_L(p) = 3p^2(1 - p) + p^3 = 3p^2 - 2p^3 \quad (1)$$

The code therefore beats an unencoded qubit for every $p < 1/2$, by a factor of about $1/(3p)$ when p is small.

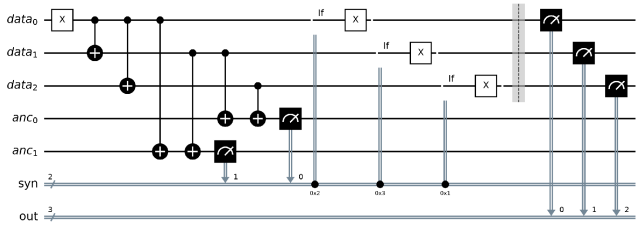


Fig. 2. The real-time circuit of Experiments 4.1 and 4.2: encoding of $|1\rangle_L$, two parity checks on ancillas, real-time correction through three conditional (if) blocks, and readout of the data qubits. Experiment 2.2 uses the same circuit with one noisy idle step after encoding. Drawn by Qiskit from the experiment code.

B. The Decoherence Deadline

A logical cycle of N syndrome rounds should finish within the dephasing time T_2, so each round's classical work, from readout to dispatching the correction, has the budget

$$\tau_{DD} = \frac{T_2}{N} = \frac{120\ \mu s}{3} = 40\ \mu s \quad (2)$$

for the project defaults $T_2 = 120$ µs and N = 3. This T_2 is deliberately conservative: the calibrated replicas in Section V have median T_2 of 144 µs and 170 µs. A Ramsey experiment measures how much phase coherence a qubit prepared in $|+\rangle$ keeps after an idle time t [2]:

$$F(t) = 0.5 + 0.5\, exp(-t / T_2) \quad (3)$$

C. The Violation Rate

For measured decode latencies $L_1, \ldots, L_n$, the violation rate at deadline τ is the fraction of cycles that miss it:

$$VR(\tau) = P(L > \tau) \approx \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}[L_i > \tau] \quad (4)$$

The deadline τ^* at which $VR(\tau^*) = 1$ percent is the 99th percentile of the latency distribution, and the headroom 40 µs / τ^* states how many times over the budget is met. Comparing VR with the roughly 1 percent surface-code threshold [4]

treats a missed deadline as one more error event. The threshold is a per-operation physical error rate, so it serves here as an error budget, not as a law that latency must obey.

IV. Methods

A. What Is Measured

Fig. 1 places the measurement in the feedback loop. The study times only the decode step: the interval from having a syndrome to having a correction, on a general-purpose host. Readout, data transfer, and dispatch of the correction are outside the measurement, so the measured latency is a lower bound on the classical part of a real loop. The device experiments then ask the converse question: how much logical error does a given amount of classical delay cause?

B. Software and Host

All experiments use Qiskit 2.5.2, Qiskit Aer 0.17.2, and qiskit-ibm-runtime 0.50.0 [18], with SciPy 1.17.1 [22], NumPy 2.4.6 [23], statsmodels 0.15.0 [24], and Matplotlib 3.11.2 [25]. The reference run used Python 3.12.3 and GCC 13.3.0 with -O2 on a Linux 6.18 virtual machine with two vCPUs of an Intel Xeon processor at 2.1 GHz. Python timings use time.perf_counter_ns, which on this host is clock_gettime(CLOCK_MONOTONIC) with 1 ns resolution. Every timing result is stored together with the Python version, operating system, CPU model, and clock details of the machine that produced it.

C. Noise Models and Device Replicas

The default noise model applies thermal relaxation with T_1 = 250 µs and T_2 = 120 µs to 50 ns idle and single-qubit gates and to 500 ns CNOT gates. Experiment 1.2 uses a single Pauli channel on one idle gate, and Experiment 2.2 uses code-capacity noise: an independent X error with probability p on each data qubit, with ideal checks. The device experiments use FakeKingston, a 156-qubit IBM Heron r2 snapshot, and FakeSherbrooke, a 127-qubit IBM Eagle r3 snapshot, whose coupling maps, basis gates, and qubit properties come from real systems [20]. Table I compares them with the project model. Because both devices have longer T_2 than the default, their own deadlines are 48 µs and 57 µs, and the 40 µs deadline is conservative.

TABLE I
Noise Parameters of the Project Model and the Device Replicas

Parameter	Project model	Heron r2	Eagle r3
Qubits	—	156	127
Median T_1 / T_2 (µs)	250 / 120	283 / 144	278 / 170
$\tau_{DD} = T_2 / 3$ (µs)	40	48	57
Single-qubit gate (ns)	50	32 (√X)	57 (√X)
Two-qubit gate (ns)	500 (CNOT)	68 (CZ)	533 (ECR)
Readout (µs)	—	2.28	1.22
Two-qubit / readout error (%)	—	0.18 / 0.95	0.78 / 1.98

Replica values are medians over all qubits of the snapshots distributed with qiskit-ibm-runtime 0.50.0. Live devices drift from them.

D. Experiments

Table II lists the experiments. They run in four stages: the physical layer, correctness and benefit of the code, decoder latency, and the protocol on device replicas. Every random draw is seeded.

TABLE II
Experiments and Sample Sizes

ID	Measurement	Sample size
1.1	Ramsey decay of $\|+\rangle$; fit of T_2	25 idle times × 4,096 shots
1.2	Pauli sensitivity of $\|+\rangle$	3 Paulis × 11 p × 8,192 shots
2.1	Syndrome decoding under noise; in-cycle Python decode time	10,000 single-shot cycles
2.2	Logical versus physical error	11 p × 100,000 shots
3.1	Shape and tail of the 2.1 latencies	10,000 samples
3.2	Decode latency of three tiers; timer floors; VR(τ)	10,000 cycles per tier, plus a repeat study of 10 × 10,000
4.1	Bare versus encoded fidelity; offline versus real-time correction	3 circuits × 4,000 shots per device
4.2	Logical error versus decoder wait (0–80 µs)	9 waits × 2 circuits × 4,000 shots per device

Experiment 3.2 discards 1,000 untimed warm-up cycles per tier. Experiments 4.1 and 4.2 run on both replicas.

E. Latency Tiers and the Timer Floor

The Python tier times a dictionary lookup between two perf_counter_ns calls. The C tier implements the same table in a shared library called through ctypes; each lookup is timed inside C with clock_gettime, so Python overhead is excluded. A third tier model, without measuring, an ARM Cortex-M7 microcontroller at 480 MHz with a 41-cycle worst-case execution time (85.4 ns) plus 5 percent Gaussian jitter. It is shown for scale only and is excluded from every statistical test.

At these speeds, reading the clock is a large part of the measurement. For both measured tiers the timer floor, two consecutive clock reads with nothing between them, is therefore recorded as a control, and the net decode cost is the median latency minus the median floor. A repeat study adds ten further runs of 10,000 cycles per measured tier, and the whole timing experiment was repeated as six complete runs on two host CPU models (Section V-H).

F. Device-Replica Protocol

All device circuits start from logical $|1\rangle$. The bare circuit holds one physical qubit for exactly the duration of the encoded circuit's encode-and-syndrome stage, 3.15 µs on Heron and 6.76 µs on Eagle, using a delay instruction. The offline circuit measures the syndrome without feed-forward and is decoded afterwards in software, shot by shot; the real-time circuit applies the correction with conditional gates before the data qubits are read out. Four protocols follow: bare qubit, majority vote ignoring the syndrome, offline decoding with majority vote, and real-time feed-forward with majority vote. All three circuits are pinned to one physical layout, and the bare qubit is the physical qubit of data qubit 0. Pauli

twirling and dynamical decoupling are disabled, and each circuit runs for 4,000 shots through the client-side Sampler introduced in qiskit-ibm-runtime 0.50.0 [26].

The replicas attach noise to instructions only, so the classical feedback window is invisible to them. The latency sweep therefore inserts an explicit delay of 0 to 80 µs between the syndrome readout and the correction, as a stand-in for decoder latency, and compares it with a bare qubit idling for the same additional time.

G. Statistical Analysis

Fidelities and error rates carry Wilson 95 percent intervals [27], which keep close to nominal coverage where the textbook Wald interval does not [28]. VR(τ) uses exact Clopper–Pearson intervals [29], because a bootstrap interval collapses to [0, 0] when no cycle misses. τ* carries a percentile-bootstrap interval with 5,000 resamples [30]; a quantile is a setting where the bootstrap behaves well. Distribution shape is assessed with the Anderson–Darling statistic on log-latency [31] and with the Kolmogorov–Smirnov distance D against fitted log-normal, gamma, and Weibull distributions. Because the parameters are fitted to the same data, D is used to rank families rather than as a p-value. The tail index α is estimated with the Hill estimator on the k largest observations $X_{(1)} \geq \ldots \geq X_{(k+1)}$, with k equal to 5 percent of the sample [32]:

$$\alpha = \frac{k}{\sum_{i=1}^{k} \ln \frac{X_{(i)}}{X_{(k+1)}}} \quad (5)$$

Tails are called light for $\alpha > 4$, moderate for $2 < \alpha \leq 4$, and heavy for $\alpha \leq 2$. Python and C latencies are compared with the Mann–Whitney U test [33] and the rank-biserial effect size

$$r_{rb} = \frac{2U_1}{n_1 n_2} - 1 \quad (6)$$

where $r_{rb} > 0$ means that Python tends to be slower. Protocols are compared with χ^2 tests on 2 × 2 success tables, and pooled miss counts with Fisher's exact test [34]. Simulator seeds are spread with NumPy's SeedSequence: when Aer simulates shot by shot, as it does for the code circuits, it seeds shot k with seed + k, so nearby integer seeds replay the same random stream one shot apart.

V. Results

A. Physical Layer

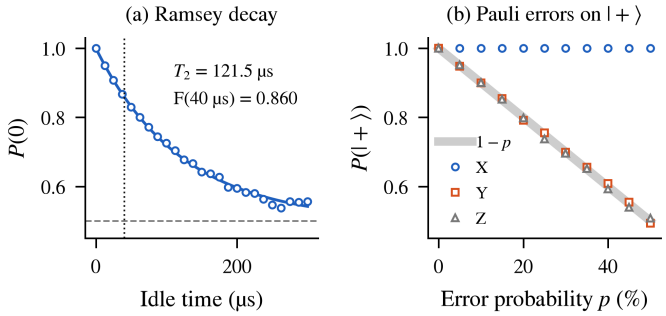


Fig. 3. (a) Ramsey decay of |+⟩ under thermal relaxation, 25 idle times with 4,096 shots each, and the fitted model; the dotted line marks the 40 µs deadline. (b) Survival of |+⟩ under one Pauli channel, 11 error rates with 8,192 shots each, against the theory line 1 − p.

The Ramsey fit recovers T_2 = 121.5 ± 1.3 µs against the 120 µs model input, 1.2 standard errors away (Fig. 3a), and the residuals are consistent with shot noise (reduced χ^2 = 1.03). At the 40 µs deadline the fidelity is 0.860, and after a full three-round cycle of 120 µs it falls to 0.686, which is why the deadline divides T_2 rather than spending it in one round. The experiment only works because the qubit starts in |+⟩. A qubit prepared in |0⟩ never decays under thermal relaxation; an early version of this experiment made that mistake, and its fit ran to the optimizer's bound at T_2 = 44,142 µs.

The |+⟩ state is immune to X errors, surviving with probability 1.000 at all 11 error rates, while Y and Z errors flip it with probability p (Fig. 3b). Every simulated point lies within 0.0123 of the theory line 1 − p, and the largest deviation is 2.6 standard errors at one of 22 points. The bit-flip code therefore corrects exactly the error that |+⟩ cannot feel and misses the errors that harm it. A code that protects superpositions needs phase checks as well, as in the Shor and surface codes, and the code experiments below protect the basis state $|1\rangle_L$.

B. Syndrome Decoding Under Noise

In Experiment 2.1, each of 10,000 cycles injected no error or one random X error, ran one noisy shot, and decoded the syndrome. The decoder chose the matching correction 9,861 times: 98.61 percent, with a Wilson interval of 98.36–98.82 percent (Table III).

TABLE III
Syndrome Decoding Under Thermal-Relaxation Noise

Injected / decoded	None	q_0	q_1	q_2	Correct
None	**2,499**	12	11	12	98.62%
q_0	15	**2,482**	23	7	98.22%
q_1	10	7	**2,459**	10	98.91%
q_2	15	8	9	**2,421**	98.70%

10,000 cycles; rows give the injected error and columns the decoded correction.

The 139 failures are spread evenly over every injected case, so the look-up table has no systematic misassignment. They come from thermal relaxation during the six 500 ns CNOT gates: a $|1\rangle$ data qubit decays or an ancilla is disturbed, and the measured syndrome no longer matches the injected error. Timed inside the full simulated cycle, the Python decode step had a median of 716 ns and a 99th percentile of 1,193 ns. One cycle in 10,000 took 104 µs, and 13 took longer than 10 µs, a first sign of the tail studied in Sections V-D and V-E.

C. Logical Versus Physical Error

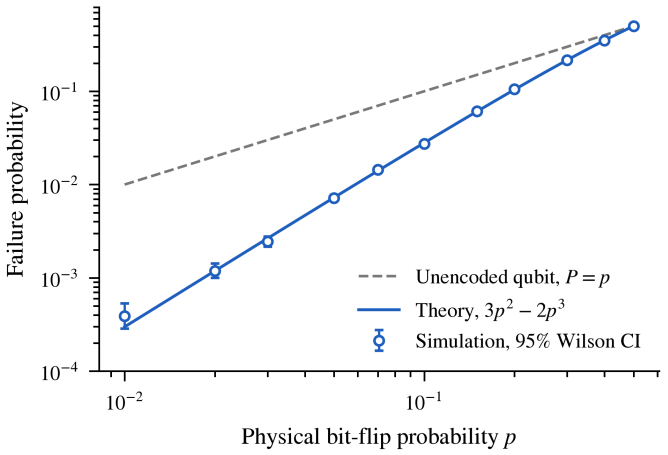


Fig. 4. Logical failure probability versus physical bit-flip probability, 100,000 shots per point, with the theory curve $3p^2 - 2p^3$ and an unencoded qubit for comparison. All 11 Wilson intervals contain the theory value.

Fig. 4 shows the code with real-time correction under code-capacity noise. All 11 simulated points contain the theory value inside their 95 percent interval, and together they fit Eq. (1) well ($\chi^2 = 12.6$ on 11 degrees of freedom, $p = 0.32$), so the code delivers the quadratic suppression that theory predicts. At $p = 1$ percent the logical failure rate is 0.039 percent (0.029-0.053 percent), against 0.986 percent for a bare qubit: 25 times fewer failures. The advantage shrinks to 3.6 times at $p = 10$ percent and vanishes at the break-even point $p = 50$ percent.

D. Decode Latency and the Violation Rate (RQ1)

Table IV summarizes the reference run. A typical decode takes 113 ns in Python and 31 ns in C, about 350 and 1,300 times inside the 40 µs budget. VR falls below 1 percent at $\tau^* =$ 268 ns for Python and 46 ns for C, a headroom of 149 and 870 times. At the deadline itself, Python missed once in 10,000 cycles, with a 41.8 µs cycle, and C never missed. The timer floors of 63 ns and 26 ns mean that the net cost of decoding is about 50 ns in Python and 5 ns in C: most of what both tiers measure is the clock.

TABLE IV
DECODE LATENCY IN THE REFERENCE RUN

Tier	Med. (ns)	Floor (ns)	τ^* (ns) [95% CI]	Max (µs)	Miss.	Hill α
Python	113	63	268 [248, 290]	41.8	1	3.95
C	31	26	46 [45, 47]	23.0	0	7.93
Bare metal (a)	85	—	95 [95, 96]	0.10	0	52.7

10,000 timed cycles per tier after 1,000 warm-up cycles. Miss. counts cycles above 40 µs. VR at 40 µs: Python 0.01 percent [0.0003, 0.056]; C and bare metal 0 percent [0, 0.037] (exact 95 percent intervals). (a) Modelled, not measured.

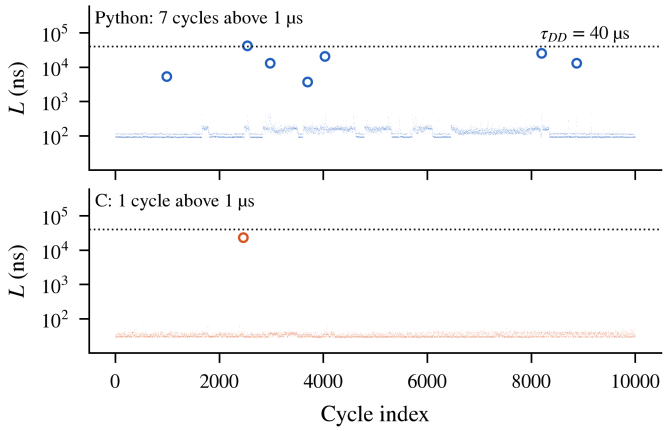


Fig. 5. Every timed cycle of the reference run in order, on a logarithmic scale. Circles mark cycles above 1 µs; the dotted line is the 40 µs deadline. The stalls are isolated events, separated by thousands of fast cycles.

Fig. 5 shows how the deadline risk arises. Almost every cycle sits near the median, and a handful jump by two to three orders of magnitude. These stalls are isolated rather than clustered. The Python baseline also steps between roughly 100 and 170 ns in blocks of a few hundred cycles; this level-shifting is visible but was not traced to a cause.

E. Shape of the Latency Tail (RQ2)

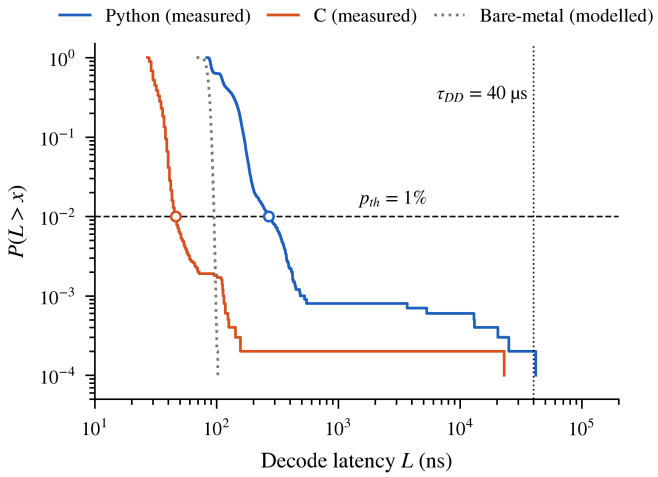


Fig. 6. Survival curves P(L > x) of decode latency in the reference run on log-log axes. Open circles mark τ^*, where VR = 1 percent. Below a survival probability of about 10^{-3}, the measured curves flatten into a plateau of stalls that the body of the distribution does not predict.

The body and near tail of the distribution are skewed and closest to log-normal by KS distance (D = 0.103 for the Experiment 2.1 data, against 0.177 for gamma and 0.333 for Weibull), although Anderson–Darling rejects exact

log-normality ($A^2 = 264$ for Python in Experiment 3.2, $p \leq 0.01$). The Hill index on the top 5 percent is 3.95 for Python, a moderate tail, and 7.93 for C, a light one.

Fig. 6 shows a second component. Below a survival probability of about 10^{-3}, the curves leave the power-law fit and flatten into a plateau. Seven Python cycles took longer than 1 µs (3.7-41.8 µs, median 13.1 µs) and one C cycle took 23.0 µs; these medians are 27 and 204 times each tier's own 99.9th percentile. Because C shows the same events without an interpreter, and the timed Python loop allocates no objects tracked by the garbage collector, the likely source is the operating system: scheduler preemption, interrupts, and hypervisor steal time on a two-vCPU virtual machine. This attribution is an inference; the events were not traced. Neither the log-normal body nor the power-law tail predicts them, so VR at large τ has to be measured directly, not extrapolated from a fit. The latency distribution is best described as a mixture: a fast body plus a rare, separate population of stalls.

F. Python Versus C (RQ4)

C is faster on almost every cycle. The Mann–Whitney test gives $p < 10^{-300}$ with $r_{rb} = +0.998$, and the ratio of medians is 3.6, or 10 after subtracting the timer floors. C does not remove the stalls: in the reference run its worst cycle (23.0 µs) is of the same order as Python's (41.8 µs). The modelled Cortex-M7 is slower than C on a Xeon in the typical case (85 ns against 31 ns) but bounded, with a worst case of 102 ns. A slower processor without an operating system in the way can thus be the safer choice for a hard deadline.

G. A Note on Interval Methods

The choice of interval matters for rare events. With one observed miss at 40 µs, the percentile bootstrap gives VR in [0, 0.03 percent], while the exact Clopper-Pearson interval is [0.0003, 0.056 percent]. The bootstrap's upper limit is 46 percent too low, because resampling cannot invent misses it never saw; with zero misses it collapses to [0, 0]. For rare-event rates, exact intervals should be used.

H. Reproducibility of the Timing Results

Seeded simulations reproduce exactly; timings do not. Table V summarizes six complete runs of the timing experiment, each with the main run and the ten-run repeat study, on two host CPU models. Pooled over 660,000 cycles per tier, Python missed the 40 µs deadline 14 times (VR = 0.0021 percent, 95 percent upper limit 0.0036 percent) and C once (0.00015 percent, upper limit 0.00084 percent). Both remain more than two orders of magnitude below the 1 percent budget. The difference between the tiers is significant (Fisher's exact test, $p < 0.001$), consistent with C's shorter timed window leaving less opportunity for a stall to land inside it.

TABLE V
TIMING RESULTS ACROSS SIX COMPLETE RUNS

Run	Python τ* (ns)	Python worst (µs)	Python misses	C τ* (ns)	C worst (µs)	C misses
R0 (ref.)	268	41.8	1	46	26.5	0
R1	216	74.3	5	40	67.4	1
R2	189	1,388.3	2	29	30.5	0
R3 (a)	399	67.2	3	110	26.0	0
R4	105	170.9	2	27	29.5	0
R5	197	42.9	1	40	37.6	0
Pooled	—	1,388.3	14	—	67.4	1

Each run covers 110,000 cycles per tier (main run plus ten-run repeat study); τ* is from the main run, and the worst cycle and misses cover all 110,000. All runs used two-vCPU Intel Xeon virtual machines at 2.1 GHz except R3 (a), at 2.8 GHz. R0 used Python 3.12.3 and the others 3.13.16. R4 and R5 are clean reproductions from the public repository.

The single slowest cycle is not a reproducible quantity. It ranged from 41.8 µs to 1.39 ms in Python and from 26.0 µs to 67.4 µs in C, and in one run a C cycle exceeded the deadline. Medians and τ* also moved between runs: Python medians spanned 73–227 ns and τ* 105–399 ns, and C medians 21–35 ns and τ* 27–110 ns. The reference run's 4–42 µs stall range is therefore a property of that run, not a bound. Across the six main runs, Python had 41 cycles above 1 µs in 60,000, about seven per 10,000.

I. Protocol on Calibrated Device Replicas (RQ3)

On both replicas, the transpiled circuit has 12 native two-qubit gates (CZ on Heron, ECR on Eagle) for the six logical CNOTs. The parity checks form a triangle of interactions, which the heavy-hexagon lattice lacks [19], so routing adds two SWAP gates. Circuit depth is 22 for the offline and 26 for the real-time circuit on Heron, and 43 and 46 on Eagle, on physical qubits 108, 109, 110, 118, 129 and 41, 53, 59, 60, 61 respectively.

TABLE VI
LOGICAL |1⟩ FIDELITY ON CALIBRATED REPLICAS

Protocol	Heron r2 (%)	Eagle r3 (%)
Bare qubit	97.88 [97.38, 98.28]	97.38 [96.83, 97.83]
Majority vote only	99.72 [99.51, 99.85]	99.28 [98.96, 99.49]
Offline decode	99.70 [99.48, 99.83]	99.10 [98.76, 99.35]
Real-time feed-forward	99.55 [99.29, 99.72]	99.00 [98.64, 99.26]

4,000 shots per circuit; brackets are Wilson 95 percent intervals.

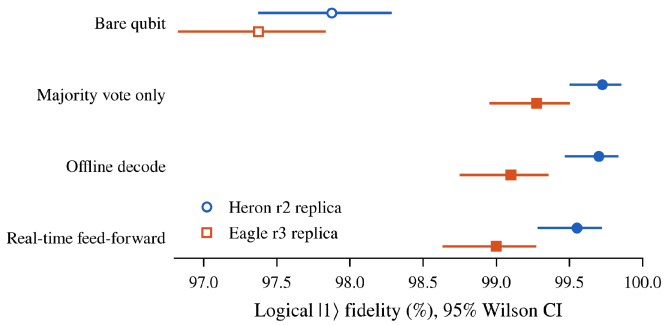


Fig. 7. Logical |1⟩ fidelity by protocol on the two device replicas, with Wilson 95 percent intervals. Open markers show the bare qubit and filled markers the encoded protocols.

Encoding helps clearly on both devices (Table VI, Fig. 7). Real-time correction beats the bare qubit by 1.68 percentage points on Heron ($\chi^2 = 42.8$, $p = 6 \times 10^{-11}$) and by 1.62 points on Eagle ($\chi^2 = 28.8$, $p = 8 \times 10^{-8}$). Real-time and offline correction are statistically tied, with differences of −0.15 points on Heron ($p = 0.36$) and −0.10 points on Eagle ($p = 0.73$). That is expected from replicas that add no noise during the feedback window.

Majority voting alone matches or beats syndrome decoding (99.72 against 99.70 percent on Heron, 99.28 against 99.10 percent on Eagle). In a single round that ends by reading out the data qubits, a majority vote already undoes any single flip, and an ancilla readout error can only cause a wrong correction. The syndrome adds value only when the logical state must survive further rounds. An early version that let the transpiler place the offline and real-time circuits on different qubits showed a spurious −0.42-point gap on Eagle, which vanished once one layout was pinned.

J. Logical Error Versus Decoder Wait

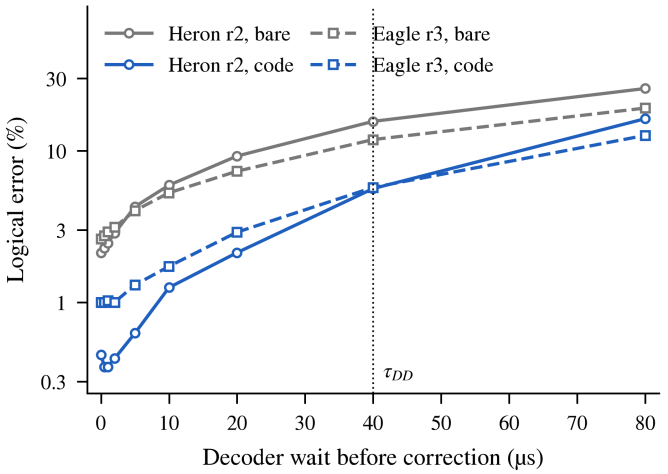


Fig. 8. Logical error versus an inserted decoder wait on the two replicas, for the encoded circuit and for a bare qubit idling for the same time. The dotted line marks the 40 µs deadline.

Fig. 8 makes the feedback window visible to the noise model. The encoded logical error starts at 0.45 percent on Heron and 1.00 percent on Eagle and doubles after waits of 7.6 µs and 12.8 µs. At 40 µs it reaches 5.6 and 5.7 percent, 12.5 and 5.7 times its zero-wait value. On Heron the error is already 1.4 times its zero-wait value after 5 µs and 2.8 times after 10 µs. The code still beats a bare qubit idling for the same time at every wait, by a factor of 2.8 on Heron at 40 µs (5.6 against 15.6 percent).

Combining this curve with the measured stalls estimates the damage they do. Interpolating the Heron sweep on a log scale and weighting it by the seven Python stalls of the reference run adds about 0.0012 percentage points of logical error per cycle on average, 0.26 percent of the 0.45 percent baseline. A single cycle hit by a 23–42 µs stall, however, would see its logical error rise five- to thirteen-fold on Heron and three- to six-fold on Eagle. The 1.39 ms stall of run R2 lies far beyond the sweep and is longer than T_2 itself.

VI. Discussion

A. The Deadline Is a Ceiling, Not a Target

By its own criterion, software decoding passes comfortably: pooled VR at 40 µs is at most 0.0036 percent for Python. The latency sweep tells a different story. A correction that arrives "on time" at 30 µs has already lost most of the code's benefit. A more useful budget is the wait at which logical error grows by a tolerable factor. Keeping that growth below about 40 percent on the Heron replica requires the whole feedback loop, including readout, data transfer, decoding, and dispatch, to finish within about 5 µs.

B. Average Cost Versus Tail Risk

Averaged over cycles, stalls are negligible. The risk lies in their concentration, the same lesson that large online services learned about tail latency [7]. At about seven stalls above 1 µs per 10,000 Python cycles, a computation of a million cycles would meet several hundred of them, and nothing in a general-purpose operating system bounds the longest one, as run R2 shows. Real-time control needs a guaranteed worst case, which is why production decoders run on FPGAs, application-specific circuits, or dedicated cores beside the control electronics [5], [16].

C. What Changing the Language Buys

Moving the decoder from Python to C cuts the typical decode 3.6-fold and, over 660,000 cycles, reduces deadline misses from 14 to 1. It does not change the nature of the risk: both tiers experience stalls of tens of microseconds, and C also missed the deadline once. For deadline compliance, the next step after C is a quieter machine, such as an isolated core, a real-time kernel [21], or dedicated hardware, rather than a faster language.

D. Simulation Versus Device Noise (RQ3)

Calibrated noise preserves the code's benefit, with gains of 1.68 and 1.62 points that are significant at $p < 10^{-7}$. What the replicas cannot show is the controller's own feed-forward delay, the very quantity this study is about. The latency sweep is a proxy for it, and a run on physical hardware is the measurement that would replace the proxy.

E. Methodological Lessons

Five choices changed results enough to matter for anyone repeating this work. Exact binomial intervals are needed for rare events, as Section V-G shows. Seeds must be spread, because nearby Aer seeds share random streams and produced correlated "independent" results. Layouts must be pinned, because qubit placement alone moved one comparison by 0.4 points. A time-matched baseline must use a delay instruction, because the transpiler deletes identity gates used as padding. And the timer floor must be subtracted, because most of what both tiers measured was the clock.

VII. Threats to Validity

Construct validity. The measured latency covers the decode step only. Readout, transfer to the classical processor,

and dispatch of the correction, which dominate a real feedback loop, are outside the measurement, so the results are a lower bound on classical latency. On the replicas, decoder delay is represented by an inserted wait, because the simulator adds no noise during the feedback window. Comparing VR with a threshold is a convenient budget, not a derived bound.

Internal validity. Stalls are attributed to the operating system by elimination, not by tracing. Kolmogorov–Smirnov p-values are optimistic after fitting, and the Hill index depends on the chosen tail fraction; the conclusions therefore rest on survival curves and direct miss counts rather than on either statistic alone. Seeded simulations reproduce bit for bit only on the same CPU type: on the 2.8 GHz host, the Eagle sweep moved by one shot in 4,000 per point, and the unmodified original code gave the same shift, so the difference comes from CPU-dependent floating-point behavior in the simulator.

External validity. All timing comes from two-vCPU cloud virtual machines on two CPU models; stall frequency and length depend strongly on the host, its load, and its kernel, so a laptop or a dedicated server will differ. The replicas are fixed snapshots, whereas real devices drift. The bit-flip code corrects only X errors, runs a single round, and here protects a basis state. Matching and union-find decoders for the surface code cost far more per round [13], [14], so the margins found here do not transfer to them. No physical quantum processor was used.

VIII. Reproducibility and Artifact

A. Software Artifact

The public repository [35] contains the experiment code (seven experiment scripts, one analysis script, two helper modules, and the C decoder), the reference run with its raw per-cycle latencies, summaries of all six timing runs, and pinned requirements. Its entry point is an executed Jupyter notebook [36] that presents the theory, runs every experiment through the same modules, and draws every figure. Because notebooks often fail to rerun [37], it follows published advice: pinned dependencies, relative paths, seeded randomness, and outputs kept apart from inputs [38]. A single-page walkthrough, published with the repository, is generated from the source files, so the code it displays cannot diverge from the code that ran. One script regenerates every figure in this paper and the pooled numbers of Table V from the committed files, and the manuscript itself is built from versioned sources.

B. Verification

Automated tests check the parts of the pipeline whose correct answers are known: the noiseless syndrome for every combination of logical state and injected flip, the real-time circuit returning $|111\rangle$ without noise, agreement between the C and Python decoders under strict compiler warnings, recovery of a known Pareto tail index by the Hill estimator, the exact zero-miss bound, and the shared layout of the replica circuits. Synthetic data appear only in these estimator checks; every reported result comes from simulated circuits and measured timings.

The notebook adds independent checks. It refits the Ramsey data; recomputes the Clopper–Pearson intervals, the Mann–Whitney statistic, and the Hill index from first principles; derives (1) by enumerating all eight error patterns; demonstrates each pitfall of Section VI-E; runs the tests; compares the seeded results with the reference run; and confirms that the walkthrough, the README, and this manuscript match the data. It stops with an error if any check fails, and its final execution for this manuscript passed every check.

For this manuscript, the repository was reproduced from a clean environment several times on November 11, 2026. Two runs, from a fresh clone of the public repository and from the final revision under Python 3.13.16, completed the full pipeline without errors or warnings in about six minutes, matched all seven seeded result files byte for byte, and passed all tests; both contribute to Table V. The final package was then set up from scratch and the notebook executed under Python 3.13.16 and 3.11.17, again with every check passing and every seeded file identical; those later timings are not pooled.

C. Reproduction Procedure

On Linux, macOS, or Windows Subsystem for Linux with Python 3.11–3.13 and GCC, running bash setup.sh creates a virtual environment with the pinned versions. Executing the notebook then reproduces and verifies the whole study in about five minutes on two cores; alternatively, bash run_all.sh reruns every experiment into a separate output folder and pytest runs the tests. No IBM account is needed for any result in this paper. The submit and retrieve commands of the hardware script run the same three circuits on the least-busy physical device that supports dynamic circuits; the job of 12,000 shots fits within the free IBM Quantum Open Plan allowance of 10 minutes per 28 days.

IX. Conclusion and Future Work

Software decoding of the three-qubit bit-flip code meets a 40 µs deadline in all but about two cycles per 100,000 in Python, and fewer in C. The remaining risk comes from operating-system stalls, which reached 1.39 ms and which no fitted distribution predicts, rather than from decoder speed. On a calibrated IBM device, replicating the code with real-time feed-forward gains 1.6 to 1.7 percentage points of fidelity over a bare qubit, but its logical error doubles after only 8 to 13 µs of decoder wait, far below the deadline. Table VII answers the four research questions.

TABLE VII
Answers to the Research Questions

Question	Answer
RQ1: Where does VR(τ) cross 1 percent?	At $\tau^* = 268$ ns (Python) and 46 ns (C) in the reference run. At 40 µs, pooled VR ≤ 0.0036 percent (Python) and ≤ 0.00084 percent (C).
RQ2: Is the tail light or heavy?	Two-part: a skewed, roughly log-normal body with Hill $\alpha = 3.95$ (Python) and 7.93 (C), plus a separate plateau of stalls of tens of microseconds to 1.39 ms.

Question	Answer
RQ3: Does device noise change the benefit?	The benefit survives: +1.68 and +1.62 points ($p < 10^{-7}$). Each microsecond of wait erodes it; the error doubles after 7.6–12.8 µs.
RQ4: How much does C help?	3.6 times on the median (10 times net of the timer floor) and fewer misses (1 against 14 in 660,000 cycles), but stalls remain.

Five steps would carry the work forward: run the protocol on a Heron processor to measure the true cost of real-time feed-forward; extend it to several syndrome rounds on a superposition state, where real-time correction matters and majority voting no longer suffices; time the whole feedback loop with the controller's own timestamps; repeat the latency study on an isolated core with a real-time kernel to test whether the stall plateau disappears; and replace the look-up table with a surface-code matching decoder to measure how its latency tail grows with code distance.

Data and Code Availability

All code, the executed notebook, reference results, timing-run summaries, figure scripts, and the manuscript source are available in the project's GitHub repository [35], and the walkthrough page is at https://requiemdoomed.github.io/QEC-Latency-Decoder/.

References

[1]B. M. Terhal, "Quantum error correction for quantum memories," *Rev. Mod. Phys.*, vol. 87, no. 2, p. 307, 2015, doi: 10.1103/RevModPhys.87.307.

[2]P. Krantz, M. Kjaergaard, F. Yan, T. P. Orlando, S. Gustavsson, and W. D. Oliver, "A quantum engineer's guide to superconducting qubits," *Appl. Phys. Rev.*, vol. 6, no. 2, Art. no. 021318, 2019, doi: 10.1063/1.5089550.

[3]E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, "Topological quantum memory," *J. Math. Phys.*, vol. 43, no. 9, pp. 4452–4505, 2002, doi: 10.1063/1.1499754.

[4]A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Phys. Rev. A*, vol. 86, no. 3, Art. no. 032324, 2012, doi: 10.1103/PhysRevA.86.032324.

[5]F. Battistel et al., "Real-time decoding for fault-tolerant quantum computing: Progress, challenges and outlook," *Nano Futures*, vol. 7, no. 3, Art. no. 032003, 2023, doi: 10.1088/2399-1984/aceba6.

[6]Google Quantum AI and Collaborators, "Quantum error correction below the surface code threshold," *Nature*, vol. 638, no. 8052, pp. 920–926, 2025, doi: 10.1038/s41586-024-08449-y.

[7]J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013, doi: 10.1145/2408776.2408794.

[8]P. W. Shor, "Scheme for reducing decoherence in quantum computer memory," *Phys. Rev. A*, vol. 52, no. 4, pp. R2493–R2496, 1995, doi: 10.1103/PhysRevA.52.R2493.

[9]M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th anniversary ed. Cambridge, U.K.: Cambridge Univ. Press, 2010.

[10]C. Ryan-Anderson et al., "Realization of real-time fault-tolerant quantum error correction," *Phys. Rev. X*, vol. 11, no. 4, Art. no. 041058, 2021, doi: 10.1103/PhysRevX.11.041058.

[11]S. Krinner et al., "Realizing repeated quantum error correction in a distance-three surface code," *Nature*, vol. 605, no. 7911, pp. 669–674, 2022, doi: 10.1038/s41586-022-04566-8.

[12]N. Sundaresan et al., "Demonstrating multi-round subsystem quantum error correction using matching and maximum likelihood decoders," *Nat. Commun.*, vol. 14, Art. no. 2852, 2023, doi: 10.1038/s41467-023-38247-5.

[13]N. Delfosse and N. H. Nickerson, "Almost-linear time decoding algorithm for topological codes," *Quantum*, vol. 5, Art. no. 595, 2021, doi: 10.22331/q-2021-12-02-595.

[14]O. Higgott and C. Gidney, "Sparse Blossom: Correcting a million errors per core second with minimum-weight matching," *Quantum*, vol. 9, Art. no. 1600, 2025, doi: 10.22331/q-2025-01-20-1600.

[15]L. Skoric, D. E. Browne, K. M. Barnes, N. I. Gillespie, and E. T. Campbell, "Parallel window decoding enables scalable fault tolerant quantum computation," *Nat. Commun.*, vol. 14, Art. no. 7040, 2023, doi: 10.1038/s41467-023-42482-1.

[16]B. Barber et al., "A real-time, scalable, fast and resource-efficient decoder for a quantum computer," *Nat. Electron.*, vol. 8, no. 1, pp. 84–91, 2025, doi: 10.1038/s41928-024-01319-5.

[17]A. D. Córcoles et al., "Exploiting dynamic quantum circuits in a quantum algorithm with superconducting qubits," *Phys. Rev. Lett.*, vol. 127, no. 10, Art. no. 100501, 2021, doi: 10.1103/PhysRevLett.127.100501.

[18]A. Javadi-Abhari et al., "Quantum computing with Qiskit," 2024, arXiv:2405.08810.

[19]C. Chamberland, G. Zhu, T. J. Yoder, J. B. Hertzberg, and A. W. Cross, "Topological and subsystem codes on low-degree graphs with flag qubits," *Phys. Rev. X*, vol. 10, no. 1, Art. no. 011022, 2020, doi: 10.1103/PhysRevX.10.011022.

[20]IBM Quantum, "Fake provider," qiskit-ibm-runtime API documentation. Accessed: Nov. 11, 2026. [Online]. Available: https://quantum.cloud.ibm.com/docs/api/qiskit-ibm-runtime/fake-provider

[21]F. Reghenzani, G. Massari, and W. Fornaciari, "The real-time Linux kernel: A survey on PREEMPT_RT," *ACM Comput. Surv.*, vol. 52, no. 1, pp. 1–36, 2019, doi: 10.1145/3297714.

[22]P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nat. Methods*, vol. 17, no. 3, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[23]C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[24]S. Seabold and J. Perktold, "Statsmodels: Econometric and statistical modeling with Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 92–96, doi: 10.25080/Majora-92bf1922-011.

[25]J. D. Hunter, "Matplotlib: A 2D graphics environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.

[26]IBM Quantum, "Qiskit Runtime client release notes." Accessed: Nov. 11, 2026. [Online]. Available: https://quantum.cloud.ibm.com/docs/api/qiskit-ibm-runtime/release-notes

[27]E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927, doi: 10.1080/01621459.1927.10502953.

[28]L. D. Brown, T. T. Cai, and A. DasGupta, "Interval estimation for a binomial proportion," *Statist. Sci.*, vol. 16, no. 2, pp. 101–133, 2001, doi: 10.1214/ss/1009213286.

[29]C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934, doi: 10.1093/biomet/26.4.404.

[30]B. Efron, "Bootstrap methods: Another look at the jackknife," *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979, doi: 10.1214/aos/1176344552.

[31]T. W. Anderson and D. A. Darling, "A test of goodness of fit," *J. Amer. Statist. Assoc.*, vol. 49, no. 268, pp. 765–769, 1954, doi: 10.1080/01621459.1954.10501232.

[32]B. M. Hill, "A simple general approach to inference about the tail of a distribution," *Ann. Statist.*, vol. 3, no. 5, pp. 1163–1174, 1975, doi: 10.1214/aos/1176343247.

[33]H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *Ann. Math. Statist.*, vol. 18, no. 1, pp. 50–60, 1947, doi: 10.1214/aoms/1177730491.

[34]R. A. Fisher, "On the interpretation of χ^2 from contingency tables, and the calculation of P," *J. Roy. Statist. Soc.*, vol. 85, no. 1, pp. 87–94, 1922, doi: 10.2307/2340521.

[35]S. Shakya, "QEC-Latency-Decoder: How fast does a quantum error-correction decoder need to be?" version 2.1.0, GitHub repository, 2026. Accessed: Nov. 11, 2026. [Online]. Available: https://github.com/requiemdoomed/QEC-Latency-Decoder

[36]T. Kluyver et al., "Jupyter Notebooks—A publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Schmidt, Eds. Amsterdam, The Netherlands: IOS Press, 2016, pp. 87–90, doi: 10.3233/978-1-61499-649-1-87.

[37]J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, "A large-scale study about quality and reproducibility of Jupyter notebooks," in *Proc. 16th Int. Conf. Mining Softw. Repositories (MSR)*, 2019, pp. 507–517, doi: 10.1109/MSR.2019.00077.

[38]A. Rule et al., "Ten simple rules for writing and sharing computational analyses in Jupyter Notebooks," *PLoS Comput. Biol.*, vol. 15, no. 7, Art. no. e1007007, 2019, doi: 10.1371/journal.pcbi.1007007.